

Autoencoder: Complete Guide

An autoencoder is a neural network designed to **copy its input to its output** through a compressed bottleneck representation. It learns to efficiently encode and reconstruct data by discovering the most important features.

Components

1. Encoder

- **Purpose:** Compresses input into a smaller representation
- **Structure:** Multiple neural layers that gradually reduce size
- **Output:** Creates the bottleneck code
- **Function:** $h = f(x)$

2. Bottleneck

- **Purpose:** Stores the compressed representation (the “code”)
- **Structure:** **Exactly one layer** - the narrowest point
- **Key:** Forces information compression
- **Representation:** $\mathbf{h} \in \mathbb{R}^d$ where $d < \text{input dim}$

3. Decoder

- **Purpose:** Reconstructs original input from the bottleneck code
- **Structure:** Multiple neural layers that gradually expand size
- **Output:** Produces the final reconstruction
- **Function:** $r = g(h)$

Formulation

Objective Function

The autoencoder minimizes the reconstruction error:

$$L(\mathbf{x}, g(f(\mathbf{x})))$$

Where:

- $\mathbf{x}$ is the input data
- f is the encoder function
- g is the decoder function
- L is the loss function (typically Mean Squared Error)

Reconstruction Loss

For Mean Squared Error:

$$L(\mathbf{x}, \mathbf{r}) = \frac{1}{m} \sum_{i=1}^m (x_i - r_i)^2$$

Where:

- m is the number of features
- x_i are input values
- r_i are reconstructed values

Analogy

Scenario: You need to summarize a 1000-page book into 1 page, then rewrite the book from that summary.

Component	Analogy	Purpose
Encoder	Summary Writer	Extracts key points from full book
Bottleneck	1-Page Summary	Contains only essential information

Component	Analogy	Purpose
Decoder	Book Rewriter	Reconstructs book from summary
Training	Practice Sessions	Improve summary and rewriting skills

Learning Process: Through practice, the writer learns what's truly important to include in the summary for accurate reconstruction.

Example: MNIST Digit Autoencoder

Architecture Specification

Input (784) → Encoder (256) → Encoder (128) → [Bottleneck (32)] → Decoder (128)

Forward Pass Walkthrough

Step 1: Input Processing

- Input image of digit "5" (28×28 pixels = 784 values)
- $\mathbf{x} = [0.1, 0.9, 0.2, 0.8, \dots]$ (pixel intensities)

Step 2: Encoder Forward Pass

```
# Layer 1: 784 → 256
h1 = σ(W1 · x + b1) # Detects basic features (edges, corners)

# Layer 2: 256 → 128
h2 = σ(W2 · h1 + b2) # Combines features into shapes

# Bottleneck: 128 → 32
h = σ(W3 · h2 + b3) # Compressed code
```

Step 3: Bottleneck Representation

Compressed code (32 values):

$$\mathbf{h} = [0.8, 0.1, 0.9, 0.0, 0.7, \dots]$$

Feature interpretation:

- $h_1 = 0.8 \rightarrow$ “strong top curve present”
- $h_2 = 0.1 \rightarrow$ “weak diagonal line”
- $h_3 = 0.9 \rightarrow$ “bottom curve detected”
- $h_4 = 0.0 \rightarrow$ “no horizontal line”
- $h_5 = 0.7 \rightarrow$ “medium vertical line”

Step 4: Decoder Forward Pass

```
# Layer 1: 32 → 128
d1 = σ(W4 · h + b4)    # Interprets features into basic shapes

# Layer 2: 128 → 256
d2 = σ(W5 · d1 + b5)  # Combines shapes into pixel patterns

# Output: 256 → 784
r = σ(W6 · d2 + b6)    # Final reconstruction
```

Step 5: Output

- Reconstructed image of “5”
- $\mathbf{r} = [0.2, 0.8, 0.3, 0.7, \dots]$ (reconstructed pixels)

Learning Process (Backpropagation)

Training Algorithm

1. Forward Pass

```
input_image = [0.1, 0.9, 0.2, 0.8, ...] # Original "5"
bottleneck_code = encoder(input_image)    # [0.8, 0.1, 0.9, ...]
reconstructed_image = decoder(bottleneck_code) # Reconstruction attempt
```

2. Loss Calculation

$$L = \frac{1}{784} \sum_{i=1}^{784} (x_i - r_i)^2$$

Example: $L = 0.15$ (fairly bad reconstruction)

3. Backward Pass - Gradient Computation

Decoder Gradients:

$$\frac{\partial L}{\partial W_{\text{decoder}}} = \frac{\partial L}{\partial r} \cdot \frac{\partial r}{\partial W_{\text{decoder}}}$$

Message: "Your reconstruction from code [0.8, 0.1, 0.9] looked like a '3', but should be a '5'"

Encoder Gradients:

$$\frac{\partial L}{\partial W_{\text{encoder}}} = \frac{\partial L}{\partial r} \cdot \frac{\partial r}{\partial h} \cdot \frac{\partial h}{\partial W_{\text{encoder}}}$$

Message: "Your code [0.8, 0.1, 0.9] caused bad '3' reconstruction instead of '5'"

4. Weight Updates

Using gradient descent:

$$W \leftarrow W - \eta \frac{\partial L}{\partial W}$$

Where η is the learning rate.

TensorFlow Implementation

```

import tensorflow as tf
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Dense
import numpy as np

# Clear 3-component structure
def build_autoencoder(input_dim=784, bottleneck_dim=32):
    # Input layer
    input_layer = Input(shape=(input_dim,), name='input')

    # ENCODER (compression)
    encoder_l1 = Dense(256, activation='relu', name='encoder_1')(input_layer)
    encoder_l2 = Dense(128, activation='relu', name='encoder_2')(encoder_l1)

    # BOTTLENECK (exactly one layer)
    bottleneck = Dense(bottleneck_dim, activation='relu', name='bottleneck')(encoder_l2)

    # DECODER (reconstruction)
    decoder_l1 = Dense(128, activation='relu', name='decoder_1')(bottleneck)
    decoder_l2 = Dense(256, activation='relu', name='decoder_2')(decoder_l1)
    output_layer = Dense(input_dim, activation='sigmoid', name='output')(decoder_l2)

    # Build model
    autoencoder = Model(input_layer, output_layer, name='autoencoder')
    return autoencoder

# Create and compile model
autoencoder = build_autoencoder()
autoencoder.compile(
    optimizer='adam',
    loss='mse',
    metrics=['mae']
)

# Model summary
autoencoder.summary()

# Training example (with dummy data)
def train_autoencoder():
    # Generate dummy MNIST-like data
    x_train = np.random.random((1000, 784))
    x_test = np.random.random((200, 784))

```

```
# Train the model
history = autoencoder.fit(
    x_train, x_train, # Input and target are the same
    epochs=10,
    batch_size=32,
    validation_data=(x_test, x_test),
    verbose=1
)

return history

# Uncomment to train
# history = train_autoencoder()
```

Key Formulas Summary

Component	Formula	Description
Encoder	$h = f(x) = \sigma(W_3\sigma(W_2\sigma(W_1x + b_1) + b_2) + b_3)$	Multi-layer compression
Bottleneck	$\mathbf{h} \in \mathbb{R}^d$	Compressed representation
Decoder	$r = g(h) = \sigma(W_6\sigma(W_5\sigma(W_4h + b_4) + b_5) + b_6)$	Multi-layer reconstruction
Loss	$L = \frac{1}{m} \sum_{i=1}^m (x_i - r_i)^2$	Mean squared error
Gradient Update	$W \leftarrow W - \eta \frac{\partial L}{\partial W}$	Weight optimization

Regularized Autoencoders

- **The Problem:** Simply making the code layer small (undercomplete) isn't the only way to prevent cheating. Even large ("overcomplete") networks can learn nothing useful.
 - **The Solution:** Instead of just limiting size, we add a **constraint (regularizer)** to the loss function. This forces the model to have other desirable properties beyond just copying the input.
 - **Goal:** To train powerful, overcomplete autoencoders that still learn meaningful features by using a cleverly designed loss function.
 - **Types:** The main types discussed are **Sparse, Denoising, and Contractive Autoencoders**.
-

4. Sparse Autoencoders

- **Definition:** A regularized autoencoder that learns to represent data using only a **small number of active neurons** in the code layer at any time.
 - **How it Works:** A sparsity penalty term $\Omega(h)$ is added to the loss function. This penalty encourages most of the neurons in the code layer to be zero (or close to zero) for each data sample.
 - **Analogy:** Imagine summarizing a book not by writing a short summary (undercomplete), but by writing a long, detailed summary where you are only allowed to use a few key words per chapter.
 - **Result:** The network is forced to represent each input using a unique, sparse combination of features, which often leads to the discovery of interpretable features (like edge detectors in images).
-

5. Denoising Autoencoders (DAE)

- **Definition:** A regularized autoencoder that is trained to **recover the original input from a corrupted or noisy version**.

- **How it Works:** The input x is artificially corrupted (e.g., by adding noise or masking pixels) to create $\tilde{x}$. The autoencoder must then reconstruct the clean x from $\tilde{x}$.
 - **Analogy:** Teaching someone to restore a damaged photograph. They must learn what a “normal” photo looks like to fix it, rather than just memorizing it.
 - **Result:** By learning to denoise, the autoencoder is forced to understand the true underlying structure and patterns of the data, making it robust and preventing it from learning a trivial identity function.
-

6. Contractive Autoencoders (CAE)

- **Definition:** A regularized autoencoder that is trained to be **insensitive to small changes** in the input.
- **How it Works:** A penalty term Ω is added that penalizes the derivatives of the code with respect to the input. In simple terms, it forces the encoder to produce similar codes for similar inputs.
- **Analogy:** Learning to recognize a cat. You should produce the same “cat” code whether the cat is facing left or right, in bright or dim light. The model learns to be invariant to unimportant variations.
- **Result:** The model learns a smooth mapping from the input to the code, capturing the essence of the data manifold.